

Cognitive Memory for AI Agents: Preservation-First Decay, Core Promotion, and Retrieval-Driven Reinforcement

Bhekani Khumalo
Independent Researcher
hello@bhekani.com
<https://www.bhekani.com>

May 2026

Abstract

Most AI memory systems focus on extraction and retrieval, leaving the stored-memory lifecycle under-specified. We present **cognitive-memory**, an open-source agent-memory architecture that treats forgetting as reduced accessibility rather than immediate deletion. Memories decay through configurable curves, are reinforced on retrieval, can earn core status through sustained cross-session use, and migrate between hot, cold, and stub states with reversible consolidation and explicit deep recall. On LoCoMo, a v0.5 tuned configuration reaches 46.2% overall F1 and 51.3% multi-hop F1; on LongMemEval-S, a May 2026 current-refresh run reaches 71.6% task-averaged accuracy. A controlled LTI-Bench scenario recovers all identity-critical facts after 30 days, but a floors-off ablation shows unchanged critical retention, suggesting stability accumulation and softened retention weighting are more load-bearing than floor-clamping in this horizon. These results position **cognitive-memory** as a competitive single-stage memory architecture and expose open questions around longer-horizon decay, core-promotion thresholds, and associative retrieval.

1 Introduction

Large language models are stateless. Every inference is a fresh start. To build agents that maintain continuity across sessions, developers bolt on external memory systems, and a growing ecosystem of tools exists to do exactly that [Chhikara et al., 2025, Packer et al., 2023, Sumers et al., 2024]. These systems have gotten quite good at two things: extracting salient information from conversations, and retrieving relevant memories when prompted. But between extraction and retrieval lies a dead zone. Once stored, memories sit in a database with the same weight and accessibility they had on the day they were created.

This is not how biological memory works. Human memory is dynamic: it strengthens with use, weakens with neglect, and selectively preserves information that has proven important over time [Ebbinghaus, 1885, Bjork and Bjork, 1992]. Forgetting is functional rather than a failure mode. Richards and Frankland [2017] argue that the goal of memory is not faithful information transmission but rather guiding intelligent decision-making in noisy, changing environments. Partial forgetting enables generalization by stripping away context-specific details and preserving underlying patterns.

For AI agents, the engineering consequences of ignoring this middle phase are concrete. A memory system that retains everything at full strength produces context windows bloated with noise, stale facts weighted equally to fresh ones, and no way to distinguish a recurring preference from a passing remark. Consolidation (compressing old memories into summaries) is the field’s

most common response, but it is lossy and irreversible: the original memories are destroyed, and with them the ability to recover details that may become relevant again.

This paper presents an alternative approach. Our system, **cognitive-memory**, implements biologically-inspired decay that dims memories without immediately destroying them, retrieval-driven reinforcement that strengthens memories through use, and emergent core memory detection that protects critical information. The key philosophical commitment is that **a memory should not be pruned merely because it has become hard to retrieve**. Instead, memories fade to a floor: regular memories to a faint but non-zero level, core memories to a high floor that keeps them always accessible. This preserves the possibility of reactivation during the memory’s active or cold-storage window: the everyday experience of a forgotten detail flooding back when triggered by the right cue.

We contribute: (1) a formal architecture grounded in specific cognitive science mechanisms; (2) a particular combination of lifecycle design elements (decay floors, emergent core memory promotion, reversible consolidation, two-tier retrieval boosting, and tiered hot/cold storage) that is not commonly treated as a first-class memory-management layer; (3) an open-source TypeScript and Python implementation, deployed in a production chat application, **blah.chat**;¹ and (4) an empirical evaluation on LoCoMo, LongMemEval-S, and a controlled long-term-interaction benchmark (LTI-Bench) targeted at the architectural claims (Section 6).

2 Related Work

2.1 Taxonomic Foundations

The CoALA framework [Sumers et al., 2024] provides the most influential taxonomy for agent memory, distinguishing working memory (short-term, in-context) from long-term memory (episodic, semantic, procedural). CoALA draws on the SOAR cognitive architecture and identifies memory management as a key area for future development, though its focus is primarily on the structural organization of memory rather than temporal dynamics.

2.2 Memory Architectures for Agents

Park et al. [2023] introduced the generative agents architecture, where agents maintain a “memory stream” of observations and synthesize higher-level reflections over time. Their retrieval function combines recency, importance, and relevance: an early recognition that not all memories should be weighted equally. However, the recency signal is a simple time-based decay with no mechanism for retrieval-driven strengthening or differentiated decay rates.

MemGPT [Packer et al., 2023] models agent memory on an operating system’s virtual memory hierarchy, with a core memory (in-context) and archival memory (out-of-context) layer. The agent manages its own context through explicit read/write operations. This is an architectural innovation in *how* memory is organized, but it does not address the temporal dynamics of what happens to memories over time.

Mem0 [Chhikara et al., 2025] provides a unified memory layer with extraction, deduplication, and retrieval. Recent versions describe decay mechanisms where “memories strengthen when recalled and naturally decay over time if unused,” though the implementation details and decay model are not fully specified in published materials.

2.3 Forgetting Mechanisms

The most directly related work is FadeMem [Wei et al., 2026], published in January 2026. FadeMem implements Ebbinghaus-inspired exponential decay across a dual-layer (short-term/long-term) memory hierarchy, with retention modulated by semantic relevance, access frequency,

¹<https://blah.chat>

and temporal patterns. It incorporates LLM-guided conflict resolution and memory fusion, and demonstrates improvements over Mem0 and MemGPT on multiple benchmarks. Our work shares the same cognitive science inspiration but differs in several design choices detailed in Section 4.

MemoryBank [Zhong et al., 2024] incorporates the Ebbinghaus forgetting curve to model memory strength decay and supports continual memory updating and user profile construction. SAGE [Liang et al., 2024] combines the Ebbinghaus curve with linguistic knowledge to optimize memory management, applying part-of-speech priority to determine information retention. Memoria [Lee and Jang, 2023] addresses what it calls the “fateful forgetting problem” through a human-inspired memory architecture. MemOS [Kang et al., 2025] introduces TTL-based lifespan policies and state transitions (Generated → Activated → Merged → Archived → Expired).

Recent and adjacent systems. Several systems sit close to our problem framing on long-horizon conversational memory benchmarks. ENGRAM [Patel and Patel, 2025] routes turns into episodic, semantic, and procedural records; it is the closest single-stage baseline against which we compare, with our LongMemEval-S result practically indistinguishable at the reported precision. Several systems published in our evaluation and submission window exceed both ENGRAM and our result: TiMem [Li et al., 2026] reports 76.88% on LongMemEval-S via a Temporal Memory Tree that consolidates raw observations into progressively abstracted persona representations; EverMemOS [Hu et al., 2026] reports 83.0% via an engram-inspired lifecycle of trace formation, semantic consolidation, and reconstructive recollection; MAGMA [Jiang et al., 2026] retrieves across semantic, temporal, causal, and entity graphs by query-adaptive traversal; Hindsight [Latimer et al., 2025] treats memory as a structured reasoning substrate and reports substantially higher LongMemEval and LoCoMo numbers under larger or different backbone settings.

The closest adjacent work to our preservation and scaling claims is All-Mem [Lv et al., 2026], which maintains a topology-structured memory bank with non-destructive consolidation, a bounded visible surface for online retrieval, and hop-bounded expansion from active anchors to archived evidence. The shared motivation is reversible consolidation and a hot/cold split; the mechanism differs: All-Mem evolves memory topology through offline SPLIT, MERGE, and UPDATE operations, whereas **cognitive-memory** focuses on continuous retention dynamics with decay floors, retrieval-driven reinforcement, emergent core promotion, and explicit availability–accessibility scoring.

We discuss these comparisons here so that our LongMemEval-S result is not over-claimed: it is competitive with ENGRAM-class single-stage systems, not a leaderboard result as of mid-2026. Our contribution is architectural rather than benchmark-leading.

The “Memory in the Age of AI Agents” survey [Liu et al., 2025] provides the most comprehensive recent overview, categorizing forgetting as one of three branches of memory evolution alongside consolidation and updating. That survey’s taxonomy distinguishes memory by *form* (token-level, parametric, latent) and *function* (factual, experiential, working). Our system operates entirely at the token level (memories are discrete text records in an external store) and primarily serves the factual memory function, maintaining persistent knowledge about users and their preferences across sessions. This confirms that forgetting is an active area of research, though one still in its early stages with no dominant approach.

2.4 Cognitive Science Foundations

Our architecture draws on several established results from cognitive science and neuroscience:

- **Ebbinghaus forgetting curves** [Ebbinghaus, 1885, Murre and Dros, 2015]: Memory retention is often modelled by smooth decreasing curves, with the steepest drop occurring

shortly after encoding. Exponential decay is a useful engineering prior; power-law curves may better capture longer tails in some settings.

- **Spaced repetition effects** [Cepeda et al., 2006]: Memories accessed at increasing intervals develop greater long-term stability than those accessed in massed practice.
- **Spreading activation** [Collins and Loftus, 1975]: Retrieving one memory activates semantically related memories, with activation strength decreasing with associative distance.
- **Synaptic tagging** [Frey and Morris, 1997]: Certain memories receive biochemical tags during encoding that mark them for preferential long-term consolidation, independent of subsequent access patterns.
- **Adaptive forgetting** [Richards and Frankland, 2017]: The interaction between persistence and transience optimizes memory-guided decision-making. The goal of memory is not accurate recording but flexible behaviour in changing environments.
- **Intentional forgetting** [Popov et al., 2019]: Forgetting can be an effortful, resource-consuming process that actively aids memory for target items. Our system implements only passive decay rather than directed forgetting, but this work motivates the broader view that forgetting is functional rather than merely a failure of retention.
- **Availability vs. accessibility** [Tulving and Pearlstone, 1966]: Apparent forgetting often reflects a failure of retrieval (accessibility) rather than a loss of stored information (availability). Memories may be intact in storage but inaccessible without appropriate cues.
- **Complementary Learning Systems** [McClelland et al., 1995]: The brain maintains two complementary memory stores (a fast-learning, fast-decaying hippocampal system and a slow-learning, stable neocortical system) with gradual migration of memories between them.
- **Silent engrams** [Ryan and Frankland, 2022]: Forgotten memories may persist as engram ensembles that are available but inaccessible through natural retrieval cues, recoverable only through artificial stimulation. This suggests that much natural forgetting is a state change in accessibility rather than erasure.

3 Architecture

The `cognitive-memory` system operates as a middleware layer between an LLM-based agent and its vector store. When the agent produces or encounters information worth remembering, the system extracts structured memory objects, assigns initial parameters, stores them with embeddings, and manages their lifecycle over time. On retrieval, the system queries semantically, applies retention-weighted scoring, boosts accessed memories, and returns results ordered by composite relevance.

3.1 Memory Representation

Each memory is represented as a structured object with the following fields:

3.2 Decay Model

Retention is computed as a configurable forgetting curve with a non-zero floor. The default SDK configuration uses a modified exponential decay function:

Field	Type	Description
content	string	Natural language content
category	enum	episodic, semantic, procedural, core
importance	float [0,1]	LLM-assessed salience at extraction
stability	float [0,1]	Resistance to decay; increases with use
accessCount	int	Number of times retrieved
lastAccessedAt	datetime	Timestamp of most recent retrieval
createdAt	datetime	Timestamp of creation
associations	list	Links to related memories with weights

Table 1: Memory object schema. The **core** category triggers elevated decay floors and is assigned either by LLM classification at extraction or by emergent promotion (Section 3.4).

$$R(m) = \max \left(f_m, \exp \left(\frac{-\Delta t(m)}{S(m) \cdot B(m) \cdot \tau_c} \right) \right) \quad (1)$$

where:

- $R(m) \in [f_m, 1]$ is the retention score for memory m
- f_m is the *decay floor* (see Section 3.3)
- $\Delta t(m)$ is days since last access
- $S(m) \in [0, 1]$ is the stability parameter, representing resistance to decay (with a small floor in implementation to avoid division by zero for new or malformed records)
- $B(m) = 1 + 2 \cdot \text{importance}(m) \in [1, 3]$ is the importance boost (with B clamped to 3 in implementation to guard against adapters ingesting unnormalised importance)
- τ_c is the base retention time constant, dependent on memory category

We also implement a power-law variant used in the ablations in Section 6.5:

$$R_{\text{power}}(m) = \max \left(f_m, \left(1 + \frac{\Delta t(m)}{S(m) \cdot B(m) \cdot \tau_c} \right)^{-\gamma} \right) \quad (2)$$

where γ controls tail heaviness. The current benchmark comparison uses $\gamma = 1/\ln 2 \approx 1.4427$, chosen so that the power-law curve matches the exponential curve’s one-time-constant e-folding point: at $\Delta t = SB\tau_c$, retention is approximately $e^{-1} \approx 0.368$. We retain the exponential form as the default because it is simpler and has been characterised across the full headline run; the power-law result is promising but currently measured only in preliminary single-conversation ablations.

Base retention time constants are differentiated by category to reflect the different temporal profiles of memory types documented in cognitive science [Finley, 2025]. Table 2 reports the v0.5 SDK defaults used by the current Python release; earlier paper-faithful configurations used a shorter semantic time constant of 120 days.

3.3 Decay Floors: Never Zero by Score

The central design commitment of this architecture is that **no memory reaches zero by decay score alone**. The decay floor f_m enforces a minimum retention score:

$$f_m = \begin{cases} 0.60 & \text{if category}(m) = \text{core} \\ 0.02 & \text{otherwise} \end{cases} \quad (3)$$

Category	τ_c (days)	Rationale
Episodic	45	Event memories fade relatively quickly
Semantic	240	Factual knowledge persists longer; tuned from 120 in v0.5
Procedural	∞	Skills and procedures do not decay; raw retention is fixed at 1.0
Core	120	High floor dominates; base rate matters less

Table 2: Base retention time constants by memory category in the v0.5 SDK defaults. Larger values produce slower forgetting.

A regular memory at its floor (2% retention) is unlikely to surface in normal retrieval, but it is not annihilated by the scoring function. Retrieval uses a softened accessibility term, R^α with default $\alpha = 0.3$, so a floor-retained memory has $0.02^{0.3} \approx 0.309$ accessibility. With a highly specific semantic match of 0.9, its final score is therefore approximately $0.9 \times 0.309 \approx 0.278$, rather than 0.018 under linear retention weighting. We term memories below 15% retention *faint*: rarely surfaced, but not pruned by decay alone.

This stands in deliberate contrast to systems that prune memories below a threshold (Section 4 compares directly to FadeMem on this point). Our position is not that full text should be retained forever in every deployment; Section 3.9 describes cold-storage TTL and stub conversion. The stronger claim is that decay should first degrade accessibility, not abruptly erase recoverability. The cost of retaining faint text records for a configurable dormancy window is usually small; the cost of permanently losing them before their future relevance is known is hard to predict.

This mirrors one important strand of the neurobiological evidence: many apparent memory failures reflect weakened access rather than complete erasure [Wixted, 2004]. The common experience of a long-forgotten detail returning vividly when triggered by the right cue (a smell, a name, a location) is evidence that at least some memories are dimmed, not deleted.

To make this concrete: consider a user who mentions their cat’s name in session 3, then never references it again. By session 20, that memory has decayed to a faint state. In a hard-pruning system, it may be gone. In ours, it persists during its active or cold-storage window, invisible to most normal retrieval. But when the user asks in session 47, “what was my cat called?”, a query with high semantic similarity to exactly that memory can still surface it because retention is softened rather than treated as a hard gate. The retrieval boost then begins restoring its stability, and the memory re-enters active circulation. This is the behaviour the floor-and-soft-scoring design is intended to preserve before archival stub conversion occurs, though Section 6.10 shows that floor clamping alone is not the load-bearing mechanism on LTI-Bench’s 30-day window.

3.4 Core Memory Detection

Core memories are those that should remain highly accessible regardless of access patterns: a user’s name, critical allergies, fundamental preferences. We implement two complementary pathways to core status. The first is loosely analogous to synaptic tagging [Frey and Morris, 1997], in that certain memories receive preferential treatment at encoding time, though our mechanism (LLM classification) is far simpler than the biological process. The second pathway mirrors use-dependent consolidation:

Explicit tagging. At extraction time, the LLM classifies whether a memory represents identity-critical information (names, medical conditions, family relationships). If so, it is immediately assigned `category = core` with the 60% floor.

Emergent promotion. Memories that were not initially tagged as core can earn core status through sustained use. A memory is promoted to core when it meets three criteria simultaneously:

1. **High access count:** retrieved more than a configurable threshold number of times (default: 10)
2. **High stability:** the stability parameter exceeds a threshold (default: 0.85)
3. **Cross-session persistence:** the memory has been accessed across at least a configurable number of distinct sessions (v0.5 default: 2; earlier configurations used 3)

This emergent pathway reflects the cognitive science observation that core memories in humans are not all tagged at encoding. Many become central to identity through repeated retrieval and emotional reinforcement over time [Bjork and Bjork, 1992]. A user’s preference for a particular communication style might not seem identity-critical when first expressed, but after being accessed in dozens of conversations, it has functionally become core knowledge about that user. The Phase 5 LoCoMo head-to-head used a benchmark adapter that pinned the session threshold to 2 in both v0.4 and v0.5 arms, so the LoCoMo gain in Table 4 is not attributed to this threshold change.

3.5 Retrieval Scoring

When the agent queries memory, results are scored by the product of semantic relevance and a power-law accessibility transform of retention:

$$\text{score}(m, q) = \text{sim}(m, q) \times R(m)^\alpha \quad (4)$$

where $\text{sim}(m, q)$ is cosine similarity clipped to $[0, 1]$, $R(m)$ is the retention score from Equation 1, and α controls how aggressively retention suppresses retrieval (default: $\alpha = 0.3$). This means two memories with identical semantic content can produce different retrieval scores based on their temporal profiles, while faint memories remain accessible to highly specific cues.

This multiplicative interaction is the mechanism by which the system implements what Richards and Frankland [2017] describe as memory’s role in guiding intelligent decisions: recent, repeatedly-accessed information is foregrounded, while stale or rarely-used information recedes without being eliminated.

3.6 Two-Tier Retrieval Boosting

When a memory is retrieved, its temporal parameters are updated to reflect the access event. We distinguish two tiers of retrieval, inspired by the cognitive distinction between active recall and passive priming via spreading activation [Collins and Loftus, 1975]:

Direct retrieval. When a memory is returned as a primary result for a query, it receives a full boost:

$$\text{accessCount}(m) \leftarrow \text{accessCount}(m) + 1 \quad (5)$$

$$\text{stability}(m) \leftarrow \min\left(1.0, \text{stability}(m) + 0.1 \times \min\left(2.0, \frac{\Delta t(m)}{7}\right)\right) \quad (6)$$

The stability update implements a spaced repetition effect: memories accessed after a longer gap receive a larger stability increment (up to a cap at 14 days), while memories accessed in rapid succession gain less stability per access [Cepeda et al., 2006]. Stability is clamped to $[0, 1]$, so a memory that has been frequently accessed over a long period saturates at $S(m) = 1.0$. Figure 1 shows the consequence: under identical 7-day retrieval schedules, only the directly retrieved memory accumulates enough stability to cross the core promotion threshold.

Associative retrieval. When a memory is not a direct match but is *associated* with a directly retrieved memory (see Section 3.7), it receives a smaller boost:

$$\text{accessCount}(m) \leftarrow \text{accessCount}(m) + 1 \quad (7)$$

$$\text{stability}(m) \leftarrow \min\left(1.0, \text{stability}(m) + 0.03 \times \min\left(2.0, \frac{\Delta t(m)}{7}\right)\right) \quad (8)$$

This two-tier structure captures the observation that being directly recalled strengthens a memory more than being passively activated through association, while still acknowledging that any activation at all has a reinforcing effect. The illustrative simulations in Figures 1 and 2 use this paper-faithful associative boost of 0.03; the v0.5 SDK default is 0.05 after the tuning campaign reported in Section 6.

To test whether the divergence is robust to realistic variation in access patterns rather than an artefact of regular scheduling, we ran 500 Monte Carlo simulations with the same randomised schedule applied to both memories: ~ 12 Poisson-distributed retrieval events within a 90-day window, sessions every 14 days, all other parameters fixed. Direct retrieval reached core promotion in 76.8% of runs (384/500) with a median promotion day of 68 (IQR 60–76); associative retrieval reached core promotion in 0% of runs. At day 90, direct retrieval had mean stability 0.985 ± 0.052 versus 0.417 ± 0.044 for associative (Figure 2). The separation reflects the $3.3\times$ boost ratio (0.1 vs. 0.03) compounding across access events and is insensitive to their specific timing.

3.7 Associative Memory Graph

Memories form bidirectional associations when they are retrieved together or created in the same context. Each association has a weight $w \in [0, 1]$ that strengthens with co-retrieval:

$$w(a, b) \leftarrow \min(1.0, w(a, b) + 0.1) \quad (9)$$

When memory a is directly retrieved, the system checks its associations and returns any associated memory b where $w(a, b)$ exceeds a threshold (default: 0.3), provided b ’s own retention score is above a minimum. Associated memories are marked as such in the results and receive the smaller associative boost described above.

This mechanism models spreading activation [Collins and Loftus, 1975]: retrieving a concept activates related concepts, with activation strength proportional to association weight. Over time, frequently co-retrieved memories develop strong bidirectional links, while associations between rarely co-accessed memories weaken. Specifically, association weights decay exponentially based on time since last co-retrieval:

$$w(a, b) \leftarrow w(a, b) \times \exp\left(\frac{-\Delta t_w}{90}\right) \quad (10)$$

where Δt_w is days since the association was last strengthened (i.e., since a and b were last co-retrieved). The 90-day time constant means unused associations fade slowly but steadily: after 90 days without co-retrieval, an association retains approximately 37% of its weight; after 180 days, approximately 13%. Associations that fall below the retrieval threshold (0.3) no longer trigger associative retrieval from one endpoint alone. They can strengthen again if both endpoint memories are later retrieved together through direct semantic search, same-context ingestion, deep recall, or another hot-memory path.

3.8 Consolidation

The system includes a consolidation process that clusters fading memories (retention below 0.2) by semantic similarity and compresses groups of five or more into summary memories. However,

unlike typical consolidation approaches that discard originals immediately after compression, our revised policy preserves them through the tiered storage mechanism described below. Original memories are clamped to the regular floor (0.02), marked with metadata linking them to their superseding summary (`supersededBy=<summaryId>`), and migrated to cold storage. In normal retrieval, superseded originals are not eligible as direct hits. They surface only as associated children when their superseding summary is retrieved, providing a reversible path back to the raw details during the cold-storage window.

3.9 Tiered Storage

A fundamental tension exists in any memory system that avoids pruning by decay alone: the stored corpus grows monotonically. While individual text records are cheap to store, vector similarity search degrades as the candidate pool expands. We address this with a two-tier architecture inspired by the neuroscientific distinction between memory *availability* and *accessibility* [Tulving and Pearlstone, 1966].

Tulving and Pearlstone’s foundational finding demonstrated that memories can be intact in storage (available) yet fail to be retrieved (inaccessible) without appropriate cues. Recent neuroscience has reinforced this view: engrams corresponding to apparently forgotten memories persist in “silent” states that resist natural reactivation but can be recovered through artificial stimulation [Ryan and Frankland, 2022, Guskjolen et al., 2024]. At the systems level, the Complementary Learning Systems framework [McClelland et al., 1995] posits a fast-learning, fast-decaying hippocampal store that gradually transfers memories to a slower, more stable neocortical store: a two-tier architecture with migration driven by time and consolidation.

Our tiered storage implements a computational analogue. The **hot store** is a vector-indexed table supporting embedding similarity search: this is the agent’s working memory, containing all active, recently accessed, and core memories. The **cold store** is a plain key-value table in the same database, with no embedding index or normal similarity search. Cold records retain their text and embeddings, but are removed from the hot vector index; ordinary retrieval can access them only by ID through association links. Memories migrate from hot to cold when their retention has remained at floor for a configurable threshold (default: 7 consecutive days with zero retrievals). Core memories are exempt from migration. Superseded originals from consolidation are migrated to cold storage immediately.

The cold store is not a graveyard. When a hot memory is retrieved and has associations pointing to cold IDs, the system performs an $O(1)$ lookup by identifier and applies associative scoring. If a cold memory surfaces through this path and receives a retrieval boost, it is migrated back to the hot store, re-indexed for similarity search and fully reactivated. This bidirectional migration mirrors the concept of silent engrams: memories that are available but not accessible through normal retrieval, yet recoverable when the right associative cue provides a path back. Deployments can also configure a long cold-storage TTL (default: 180 days), after which a cold record is converted into a lightweight archival stub; stubs remain excluded even in deep recall. The benchmark results reported below operate within horizons shorter than this TTL.

Deep recall mode. Cold and superseded records sit in the silent-engram regime: available but not accessible through the default index. Deep recall is the artificial stimulation. The canonical `search()` function takes an optional `deep_recall: true` parameter that adds embedding-bearing cold memories and superseded originals to the candidate set through a slower scan or candidate-expansion path; stubs remain excluded, since they no longer carry searchable embeddings. Superseded cold records receive a $0.5\times$ multiplicative score penalty so they surface only on strong query matches. Implementing this as a parameter rather than a system-wide mode keeps the API stateless: the calling agent or user decides when exhaustive recall is warranted, for example when the user asks “what exactly did I say about X three months ago?” and a

consolidated summary is insufficient. By expanding the candidate set on demand, functionally forgotten information becomes accessible without polluting normal retrieval.

Scaling behaviour. To evaluate whether tiered storage bounds the hot index across realistic usage patterns, we simulated one year of operation under six scenarios varying the memory ingestion rate (2–10 memories/day), query frequency (1–12 queries/day), and results per query (3–4). Figure 3 shows the results. Across all scenarios, the hot index stabilises at 7–11% of total memories by day 365. In the moderate case (4 memories/day, 3 queries/day), the hot index settles at 124 entries (7.0% of 1,774 total). Even in the most challenging scenario (bursty ingestion at 10 memories/day with only 2 queries/day) the hot index reaches 394 (8.7% of 4,529 total) and plateaus. The consistency of this range across diverse access patterns suggests that the bounded hot index is a structural property of the architecture rather than an artefact of any particular usage profile.

Caveats. The preservation-first assumption underlying this architecture is debatable; we discuss the counterevidence from intrinsic forgetting research in Section 8.

4 Design Comparison with FadeMem

FadeMem [Wei et al., 2026] is the most closely related system, sharing our Ebbinghaus-inspired foundation. Table 3 summarises the key differences.

The central philosophical difference is about deletion timing. FadeMem treats memory removal as a feature: pruning irrelevant memories reduces storage by 45% and improves retrieval precision. Our position is that the storage cost of retaining faint text memories for a configurable dormancy window is usually modest, while the unpredictability of future relevance makes early deletion a risky optimisation. Retrieval precision is instead achieved through the multiplicative scoring in Equation 4, hot-to-cold migration of faint memories, and eventual stub conversion of long-dormant cold records.

FadeMem’s strengths relative to our system include its LLM-guided conflict resolution (which our system now adopts in deferred form, Section 8) and its memory fusion mechanism for consolidating related information.

5 Implementation

The architecture is implemented as `cognitive-memory`, an open-source TypeScript package with a Python SDK wrapper. The system is adapter-based: it defines interfaces for vector storage and embedding providers, with default implementations for common backends.

5.1 Adapter Pattern

```
interface MemoryStore {
  store(memory: Memory): Promise<void>
  query(embedding: number[], limit: number): Promise<Memory[]>
  update(id: string, fields: Partial<Memory>): Promise<void>
}

interface EmbeddingProvider {
  embed(text: string): Promise<number[]>
}
```

This allows the core decay, boosting, and association logic to remain independent of the storage backend. The system has been tested with Pinecone, pgvector, and in-memory stores.

Design Choice	FadeMem	cognitive-memory (ours)
Decay endpoint	Memories can decay to removal; hard capacity caps (1000 LTM, 500 STM)	Decay never sets retention score to zero; floors at 2% (regular) and 60% (core), with full records eventually degrading to lightweight stubs after cold-storage TTL
Memory hierarchy	Dual-layer (STM/LTM) with promotion criteria	Flat store with category-based decay rates; core as a category, not a layer
Core memory	Not implemented as an explicit category; high-importance memories receive elevated retention through scoring but without guaranteed floor protection	Dual-path: LLM tagging at extraction + emergent promotion through use
Retrieval reinforcement	Access strengthens retention	Two-tier: direct retrieval (+full boost) vs. associative drag-along (+partial boost)
Associative linking	Not described as a separate mechanism; memory fusion serves a related consolidation role	Bidirectional weighted associations with co-retrieval strengthening
Index scaling	Hard capacity caps (1000 LTM, 500 STM); excess memories pruned	Tiered hot/cold/stub storage; hot index bounded at $\sim 10\%$ of total in simulation (Figure 3)
Conflict resolution	LLM-guided classification into contradiction, update, overlap	LLM-guided detection adopting FadeMem’s taxonomy; contradicted core memories are demoted to regular floor (Section 8)
Evaluation	Benchmark-based (MSC, LoCoMo, LTI-Bench)	Production deployment plus LoCoMo, LongMemEval-S, and a controlled LTI-Bench (Section 6)

Table 3: Design comparison between FadeMem and **cognitive-memory**.

5.2 Production Deployment

The system is deployed in `blah.chat`, a consumer chat application where users interact with LLM-based agents across multiple sessions. In this deployment, the memory system extracts memories from each conversation turn via an LLM call, stores them with initial importance and category assignments, retrieves relevant memories at the start of each agent turn for injection into the system prompt, recalculates decay on each retrieval event, and checks for emergent core promotion after each retrieval cycle. An append-only audit log captures every state change (retrieval boosts, decay recalculations, association updates, core promotions, consolidation events) with memory ID, event type, old and new values, and timestamp, enabling post-hoc lifecycle analysis. This instrumentation is the basis for the production-data study planned in Section 9; the benchmark evaluation in Section 6 uses the same SDK code path.

6 Evaluation

We evaluate `cognitive-memory` on three benchmarks: LoCoMo [Maharana et al., 2024] for multi-session conversational QA, LongMemEval-S [Wu et al., 2025] for long-horizon memory tasks across six question categories, and a controlled synthetic long-term-interaction harness (LTI-Bench, Section 6.10) targeted at the architectural claims of this paper. LoCoMo and LongMemEval-S place the system at competitive but not state-of-the-art positions; LTI-Bench shows 100% retention of identity-critical facts, compared with FadeMem’s reported 82.1% critical retention on a similar synthetic decay scenario, while a targeted floors-off ablation shows that this 30-day retention result is not caused by floor clamping alone. We also report ablations on individual feature contributions, an oracle evidence-context condition, and a judge-reliability check.

6.1 Setup

Models. Memory extraction and answer generation use `gpt-4o-mini`; embeddings use `text-embedding-3-small`. LongMemEval-S and LTI-Bench both use `gpt-4o-2024-08-06` as the judge, matching LongMemEval’s official setup and keeping judge-mediated results comparable across benchmarks.

SDK and provenance. Benchmark numbers come from logged artifact families: the May 2026 current-refresh runs (LongMemEval-S, LTI-Bench, oracle, retrieval-quality, efficiency, judge-reliability, and ablations), the Phase 5 full-LoCoMo head-to-head (the LoCoMo headline), the Phase 8 LTI-Bench decay-floor ablation, and later Phase 12–15 mechanism controls. Reported numbers should be traced to the artifact paths in Code Availability rather than inferred from the package version string, since the local SDK worktree at run time differs from the public releases. The canonical retrieval API is `search()` with scoring $\text{sim}(m, q)R(m)^\alpha$ at $\alpha = 0.3$; `retrieve()` is a compatibility wrapper. Normal search excludes cold, superseded, and stub records; deep recall also includes cold and superseded records but still excludes stubs.

Hardware. All runs are on a single machine (Apple Silicon, 128 GB RAM). Throughput is bounded by OpenAI API latency rather than local compute.

Adapter configuration. The default pipeline uses exponential decay (with a separate condition for power-law), graph expansion at one hop, rerank enabled with factor three, and hybrid lexical+dense search disabled (we ablate this). LoCoMo uses top- k of 60 with dual-perspective ingestion and the Mem0-style answer prompt. LongMemEval-S uses top- k of 20 with deep recall and rerank. The Phase 5 LoCoMo comparison isolates the v0.5 default changes that matter

under the benchmark adapter: `associative_boost` from 0.03 to 0.05 and semantic base decay from 120 to 240 days. The SDK’s `core_session_threshold` also moved from 3 to 2, but the benchmark adapter already pinned it to 2 in both arms.

6.2 LoCoMo

LoCoMo is a multi-session conversational QA benchmark with 10 conversations averaging 300 turns and 9k tokens, evaluated on five question categories: single-hop, multi-hop, temporal, open-domain, and adversarial. We score using token F1 against gold answers, following the benchmark’s protocol.

Table 4 reports our headline result against published baselines and the paper-faithful v0.4 configuration. The v0.5 tuned defaults improve full-LoCoMo F1 from 44.4% to 46.2% (+1.87pp) and LLM-judge accuracy from 58.6% to 61.3% (+2.73pp) under the same harness configuration. Multi-hop F1 reaches 51.3%, approximately $1.8\times$ Mem0’s reported multi-hop F1 of 28.4% on the same benchmark. The multi-hop category is evidence-composition-heavy, requiring the system to combine evidence from multiple sessions; this is also the category where preservation-first mechanisms have the largest plausible benefit, since stale-but-relevant evidence from earlier sessions must remain retrievable. The tuning gain is not uniform by category: temporal F1 falls from 24.1% to 22.3%, while multi-hop and open-domain gains drive the overall improvement.

System	Single	Multi	Temporal	Open	Overall
Mem0 [Chhikara et al., 2025]	—	28.4	—	—	—
cognitive-memory v0.4 (paper-faithful)	35.9	46.9	24.1	48.5	44.4
cognitive-memory v0.5 (tuned)	36.1	51.3	22.3	50.4	46.2

Table 4: LoCoMo F1 (%) on the standard category 1–4 subset ($n = 1540$). The v0.4 and v0.5 rows are from the Phase 5 full-LoCoMo head-to-head artifacts in `tuning/runs/phase5/`. Mem0’s published multi-hop value is the directly comparable literature number available from Chhikara et al. [2025]; other cross-paper LoCoMo comparisons use different settings and are treated as positioning context rather than controlled baselines.

As a later protocol-split check, we also materialized a five-conversation LoCoMo development split and a five-conversation LoCoMo test split under `tuning/splits/peer_review_20260511/`. A config selected on the development split and frozen before test evaluation achieved 50.1% F1 on the held-out protocol split ($n = 843$ category 1–4 questions; question-bootstrap 95% CI [47.6, 52.6], conversation-bootstrap 95% CI [46.1, 54.9]). This is not a pristine benchmark holdout, since the full LoCoMo benchmark had already been inspected earlier, but it is a stronger check against immediate dev/test leakage than the full-corpus head-to-head alone.

We also ran an in-house NaiveRAG baseline under the same LoCoMo harness, prompt style, answer model, embeddings, and token-F1 scorer. This is not a replacement for the Phase 5 headline row, but it gives a cleaner architectural contrast than cross-paper comparisons. Against the closest apples-to-apples `cognitive-memory` v0.5 artifact in that harness (44.77% overall F1), NaiveRAG reached 43.71% overall. The category pattern is more important than the +1.06pp overall difference: `cognitive-memory` is much stronger on temporal questions (48.47% versus 20.54%), while NaiveRAG is ahead on single-hop (55.90% versus 48.77%), multi-hop (40.08% versus 35.81%), and open-domain questions (25.04% versus 23.70%). This sharpens the claim: the lifecycle architecture is most valuable for temporal and stateful memory, not a universal replacement for dense retrieval on simple lookups.

6.3 LongMemEval-S

LongMemEval-S is the Small variant of LongMemEval, with 500 questions across six task types and approximately 53 haystack sessions per question. Following the benchmark’s protocol, we use `gpt-4o-2024-08-06` as the judge and report task-averaged accuracy. LongMemEval comparisons are sensitive to backbone model, judge model, retrieval budget, and whether the system uses a single-stage memory layer or a multi-stage reasoning architecture; we therefore treat cross-paper comparisons as positioning context rather than strict leaderboard evidence.

Table 5 reports per-task and overall accuracy. We achieve 71.6% task-averaged accuracy. The closest concurrent single-stage baseline at the time of running, ENGRAM [Patel and Patel, 2025], reports 71.4%; our result is practically indistinguishable from it at the reported precision and should be read as competitive rather than a meaningful edge. Other recent or differently configured systems (TiMem [Li et al., 2026], EverMemOS [Hu et al., 2026], MAGMA [Jiang et al., 2026], Hindsight [Latimer et al., 2025], and All-Mem [Lv et al., 2026]) now exceed or complicate both ENGRAM and our result. We discuss this positioning further in Section 8.

Task	n	Accuracy (%)
single-session-user	70	85.7
single-session-assistant	56	76.8
single-session-preference	30	46.7
multi-session	133	69.9
temporal-reasoning	133	64.7
knowledge-update	78	85.9
Task-averaged	—	71.6
Overall	500	72.6
Abstention accuracy	—	90.0
Full-context baseline (published)	—	56.2
ENGRAM [Patel and Patel, 2025] (concurrent)	—	71.4
TiMem [Li et al., 2026] (recent comparison)	—	76.88
EverMemOS [Hu et al., 2026] (recent comparison)	—	83.0

Table 5: LongMemEval-S per-task accuracy and task-averaged accuracy. Our task-averaged result is competitive with concurrent ENGRAM-class single-stage memory systems and outperforms the published full-context baseline by 15.4pp; other systems and different backbone/settings exceed this. Hindsight reports substantially higher LongMemEval-family results under different settings, but is omitted from the table to avoid treating that number as a controlled LongMemEval-S baseline. The single-session-preference category is our weakest, suggesting that short-window preference extraction is a capability gap rather than a long-horizon retention issue.

6.4 Oracle Evidence Context

To separate retrieval quality from answer-generation quality, we ran an oracle evidence-context condition on LoCoMo: for each question, the ground-truth evidence sessions are provided directly as context, bypassing the memory system. Using the same Mem0-style answer prompt and `gpt-4o-mini` answer model, this condition achieves F1 of 63.9% (LoCoMo scoring) / 61.1% (Mem0 scoring). This is not a true upper bound on the task, since the answer model can still fail to use perfect evidence. It is a same-model, same-prompt evidence condition that contextualizes the tuned memory-system F1 of 46.2%, with the remaining gap attributable to a combination of retrieval quality (Section 6.7) and answer-generation noise.

6.5 Isolated Decay-Shape Sensitivity

Power-law forgetting curves often fit human memory data better than exponential at long timescales [Murre and Dros, 2015, Wixted, 2004]. We added a power-law option to the decay model and compared the two on LoCoMo conv 0 with otherwise default v6 configuration.

Power-law decay achieves F1 of 29.5% on conv 0 versus exponential’s 25.0%, a +4.6pp improvement in this controlled decay-shape run. This artifact uses conv 0 and `gpt-4o-mini`, but a narrower configuration and lower absolute-score regime than the default-pipeline ablation in Table 6, whose artifact records the Mem0 prompt, dual-perspective ingestion, deep recall, rerank factor 3, and top- k 60. The two results are therefore separate sensitivity probes, not duplicate estimates of the same intervention. We retain exponential as the default in the published SDK because both measurements are single-conversation probes and we have not characterised power-law decay across the full corpus.

6.6 Ablations

Table 6 reports preliminary per-feature ablations on LoCoMo conv 0. Each row toggles one feature relative to the v6 default configuration; F1 is on the same conversation across conditions. This is a broader default-pipeline ablation than the isolated decay-shape sensitivity run in Section 6.5, which is why the power-law absolute scores differ. In this single-conversation slice, power-law decay has the largest positive point estimate, followed by rerank and hybrid search; graph expansion is effectively neutral. These results motivate further full-corpus ablations rather than establishing causal effects across LoCoMo as a whole.

Feature	Off (F1 %)	On (F1 %)	Δ (pp)
Power-law decay (vs exponential)	42.9	46.1	+3.2
Rerank ($\times 3$)	43.8	45.7	+1.9
Hybrid search (BM25 + dense)	42.3	43.9	+1.7
Graph expansion (1 hop)	43.1	43.2	+0.0

Table 6: Preliminary per-feature ablations on LoCoMo conv 0 under the default pipeline. Power-law decay has the largest positive point estimate in this slice; reranking and hybrid search are also positive, while graph expansion is approximately neutral. These are point estimates on a single conversation; full-corpus multi-seed ablations are future work (Section 9).

Supplemental local-model architecture control. To test whether the LoCoMo behavior was only an artifact of the OpenAI answer model, we ran an off-spec architecture-control campaign on the five-conversation held-out LoCoMo-test split using LM Studio’s local `openai/gpt-oss-120b` for extraction, reranking, and answer generation, while retaining OpenAI `text-embedding-3-small` embeddings. The LLM judge was disabled, so we compare token F1 only. This is not a benchmark-spec result and is not comparable to the headline LoCoMo table, but it provides a fixed-question, fixed-prompt, fixed-retrieval-budget control under a different chat model. On the standard category 1–4 subset ($n = 843$), the full tuned system reached 0.459 overall F1, outperforming vector-only retrieval by +0.046 paired F1 (95% CI [0.019, 0.072]), heuristic defaults by +0.051 ([0.031, 0.072]), and the no-consolidation/deep-recall ablation by +0.025 ([0.003, 0.048]). The evidence is mixed or negative for several individual mechanisms under this local setup: removing reinforcement, associative graph, core promotion, or retention weighting did not reduce overall F1 (Table 7). Temporal questions show a clear advantage over vector-only retrieval (0.538 versus 0.183), while multi-hop results are mixed. We therefore read these rows as off-spec support for the full architecture over vector-only and heuristic baselines, with consolidation/deep recall

the strongest supported individual mechanism in this campaign; they do not validate every mechanism and do not replace benchmark-spec full-corpus ablations.

Row	Trial	Overall F1	Multi-hop F1	Temporal F1
Full tuned	locomo-0029	0.459	0.409	0.538
Vector-only	locomo-0026	0.413	0.435	0.183
Heuristic default	locomo-0030	0.408	0.358	0.447
No reinforcement	locomo-0031	0.471	0.411	0.562
No associative graph	locomo-0032	0.466	0.384	0.562
No consolidation/deep recall	locomo-0033	0.434	0.371	0.495
No core promotion	locomo-0034	0.479	0.436	0.548
No retention weighting	locomo-0035	0.480	0.422	0.568

Table 7: Supplemental off-spec local-model architecture control on the five-conversation held-out LoCoMo-test split (category 1–4 subset, $n = 843$). Chat-model calls used local `openai/gpt-oss-120b`; embeddings used OpenAI `text-embedding-3-small`; judge disabled. These rows are not benchmark-spec LoCoMo results and are reported only as fixed-protocol architecture controls.

6.7 Retrieval Quality

LoCoMo annotates each question with the ground-truth dialog evidence (e.g., session-turn IDs). We post-process the per-question retrieved memory contents and compute $\text{Recall}@k$: the fraction of questions where any retrieved memory at top- k contains the evidence text (Table 8). Coverage is 99.7% (1535 of 1540 questions have annotated evidence).

k	$\text{Recall}@k$ (%)
5	24.8
10	28.5
20	31.7
60	35.6

Table 8: Evidence $\text{Recall}@k$ on LoCoMo ($n = 1535$ questions with evidence annotations). Even at $k = 60$, only 35.6% of questions have any retrieved memory containing the gold evidence under lexical containment matching. This likely undercounts semantically useful paraphrases and consolidated evidence, but still indicates retrieval is a bottleneck.

The discrepancy between $\text{Recall}@60$ (35.6%) and our tuned overall F1 (46.2%) is informative but not conclusive. The lexical evidence-containment metric likely underestimates semantically useful retrieval, especially when memories are paraphrased or consolidated. At the same time, the low recall indicates that retrieval remains a major bottleneck. The oracle evidence-context condition in Section 6.4 also leaves answer-generation and answer-utilisation headroom, so the current evidence does not isolate retrieval as the sole limiting factor.

To reduce that lexical-matching caveat, a later controlled retrieval-evidence study used an LLM judge to annotate whether top-20 retrieved memories covered LoCoMo evidence IDs on a 100-question stratified held-out sample across five conditions. The full architecture led every retrieval-level metric: MRR 0.532, $\text{Recall}@10$ 0.632, $\text{Complete}@10$ 0.500, $\text{Recall}@20$ 0.690, and $\text{Complete}@20$ 0.580. The clearest advantage is ranking quality rather than raw top-20 coverage: Full versus vector-only differs by +8.7pp MRR but only +0.3pp $\text{Recall}@20$. Removing consolidation/deep recall is more damaging than vector-only retrieval ($\text{Recall}@10$ 0.517 versus Full 0.632), making deep recall the strongest supported individual mechanism in this control. An alternative-prompt re-judge on 2,000 per-memory decisions reached 96.4% raw agreement

and Cohen’s $\kappa = 0.754$, substantial agreement, but still below the threshold at which we would treat the annotations as human-validated ground truth.

6.8 Efficiency

Table 9 reports per-stage timing from the SDK trace, aggregated across the LoCoMo run. Vector search is sub-100ms; scoring is sub-millisecond. Extraction (one LLM call per ingested turn) dominates wall time at ~ 16 s per turn. Rerank and answer generation timing live outside the SDK trace (rerank in the adapter layer, answer in the eval harness) and are deployment-dependent.

Stage	Mean (ms)	p50 (ms)	p95 (ms)
Extraction (per turn)	15,611	14,750	24,500
Embedding (per turn)	375	300	800
Vector search (per query)	54	55	67
Scoring (per query)	0.61	0.58	0.71

Table 9: Per-stage timing on LoCoMo. Extraction is the dominant cost; vector search and scoring are negligible. Rerank and answer generation are not included (deployment-dependent, not in the SDK trace).

6.9 Judge Reliability

The LoCoMo headline F1 numbers depend on a token-F1 score against gold answers, which is robust on its own. LongMemEval-S and LTI-Bench are more load-bearing because they are judge-mediated. As a local sanity check, we sampled 50 question-answer pairs stratified by category and correctness (8 buckets) from the LoCoMo run and re-judged with an alternative “EQUIVALENT vs DIFFERENT” prompt formulation.

We observe Cohen’s κ of 0.879 with 94% raw agreement; 3 of 50 pairs disagreed. This supports local prompt stability for the sampled checks, but it is not a full validation of the LongMemEval-S judge distribution and does not remove broader cross-paper comparability concerns around judge model, rubric, answer format, and retrieval budget.

6.10 LTI-Bench: Architectural Test

The benchmarks above measure end-to-end accuracy on real conversational data. LTI-Bench is a controlled synthetic harness designed instead to exercise the architectural claims of this paper: decay floors, emergent core promotion, conflict-driven supersession, revival of faint memories, and time-aware retrieval.

Scenario. A 30-day single-user interaction with 28 facts ingested across 28 daily sessions. Facts are stratified by importance:

- 8 *critical* facts (identity, medical, family, residence) accessed multiple times across the window;
- 8 *contextual* facts (project, hobby, recurring appointments) with light access patterns;
- 8 *trivial* facts mentioned once and never re-accessed (testing decay floors);
- 4 *updated* facts that get superseded mid-window (testing conflict resolution).

The harness probes the system with 42 questions across eight probe types. Sessions through day D are ingested before any probe at day D fires (*time-stepped* ingestion), so probes for facts that get superseded later in the window can correctly query the original fact at the right time.

Scoring. Each probe is scored both by token F1 against a reference answer and by an LLM judge (gpt-4o-2024-08-06, the same model and CORRECT/WRONG prompt formulation as LongMemEval-S). We treat the judge as the headline metric and F1 as supplementary; substring-style scoring is unsuitable for a controlled benchmark with paraphrased answers (a memory probe that returns “April 1, 2024” against a reference of “April 1st” is substantively correct).

Results. Table 10 reports per-category accuracy and F1. Overall accuracy is 88.1% across 42 probes; F1 is 69.7%.

Probe Category	n	Accuracy (%)	F1 (%)
core_persistence	8	100.0	93.3
decay_trivial	6	100.0	64.2
contextual_retention	6	100.0	84.5
temporal_before_update	4	100.0	85.0
temporal_after_update	4	100.0	67.7
conflict	4	75.0	67.7
revival	5	60.0	42.9
associative	5	60.0	38.7
Overall	42	88.1	69.7

Table 10: LTI-Bench per-category results (n = 42 probes total, single seed, May 2026 current-refresh artifact). The architecture passes core persistence, contextual retention, direct decay probes on never-re-accessed facts, and time-aware retrieval at 100%. Conflict remains mostly correct; revival and associative retrieval are the weakest categories at 60%. Cross-fact “what do you know about X” queries often return a subset of the relevant cluster rather than the full cluster.

Architectural reading of the results. The clearest LTI-Bench finding is critical retention: 100% of identity-critical facts are recovered at probe time, against a comparison point of 82.1% reported by FadeMem on a similar synthetic decay scenario [Wei et al., 2026]. This supports the full preservation-first lifecycle in this controlled setting, but it should not be read as direct evidence that the floor clamp is the responsible mechanism. The Phase 8 ablation below shows that, on this 30-day distribution, a system with the same retrieval and stability dynamics but zero floors keeps the same critical facts retrievable.

The other architectural properties behave mostly as designed. Direct decay probes pass: 100% of trivial facts mentioned once and never re-accessed are recoverable when probed directly at day 30. Contextual retention and time-aware retrieval also pass at 100%. Conflict-driven supersession works in both directions: all four updated facts return the post-update version after day 30, and all four return the original (pre-update) version when probed during day 10–15, when that version was still current.

Two probe categories surface real weaknesses. Revival is partial (60% accuracy, 3 of 5): direct probes remain recoverable, but oblique cues still depend on retrieval phrasing and ranking. Associative retrieval is also partial (60%, 3 of 5): a query like “what do you know about my family?” surfaces one of three family-related facts rather than the full cluster. A direct probe for any individual family fact succeeds; the failure is specifically in cross-fact recall, suggesting that the current association mechanism is an architectural scaffold whose downstream retrieval benefit is not yet strongly demonstrated.

The most important caveat: emergent core promotion fires aggressively, with 66 of 85 stored memories (78%) reaching core status by day 30, including all 8 explicitly-tagged critical facts.

The 78% rate is high enough to flag in Section 8 as a threshold-tuning failure mode rather than a success criterion.

Decay-floor ablation (negative result). The strongest, now falsified, floor-specific claim is that the decay floor is what keeps critical facts retrievable. We test it directly by setting both floors to zero (`decay_floors = {"core": 0.0, "regular": 0.0}`, exposed as a `CognitiveMemoryConfig` field in v0.5.1 specifically for this ablation) and re-running LTI-Bench at $n=3$ sub-runs per arm. The result is a null: `critical_fact_retention` is 100% in both arms, `summary.decay_trivial.mean_f1` is 0.614 in both arms, and `summary.core_persistence.mean_f1` drops only -1.04pp ($0.927 \rightarrow 0.917$) at the median. These Phase 8 row-log fields are F1 aggregates, so they should be compared with Table 10’s F1 column rather than its accuracy column. Overall accuracy nominally improves under floors-off ($0.881 \rightarrow 0.905$, +2.38pp at $n=3$), but $n=3$ is too small to support a meaningful difference claim. The architectural claim survives in weaker form: floors are not the load-bearing mechanism for critical-fact retention on LTI-Bench’s 30-day window.

The design implication is that retention floors are not the only preservation mechanism in the architecture. On this distribution, stability accumulation through repeated direct retrieval keeps recently useful memories from decaying quickly, while the softened score $\text{sim}(m, q)R(m)^\alpha$ at $\alpha=0.3$ makes retention a gentle accessibility weight rather than a hard gate. High-similarity critical facts therefore remain near the top even when floor clamping is removed. Floors are designed to matter at horizons where stability decays past the floor’s clamping point; LTI-Bench’s 30-day window does not reach there often enough to make the floor mechanism necessary. A longer-horizon ablation is the cleanest way to test the strong version of the floor-specific claim, and is now Section 9 item 1. We report this null result because it materially changes the paper’s load-bearing argument: the benchmark evidence here supports the broader preservation-first lifecycle design, not floor-clamping in isolation.

The 42-probe sample is too small for strong statistical claims, and the scenario is hand-authored, so LTI-Bench is best read as a confirmatory architectural test rather than a generalisation benchmark. We report it because the benchmark categories above (LoCoMo, LongMemEval-S) measure end-to-end QA and do not directly probe the lifecycle properties this paper argues for.

7 Responsible Memory and User Control

The preservation-first stance defended in Section 3.3 has a flip side: by design, the architecture is reluctant to delete. Cold storage, reversible consolidation, and deep recall all reduce the chance that a user’s identity-bearing facts are accidentally lost, and they correspondingly increase the chance that those facts persist longer than the user wants. Agent memory systems store names, family relationships, preferences, work context, locations, medical details, and other identity-bearing information. A preservation-first architecture therefore needs product and deployment constraints, not only retrieval mechanics. The SDK treats deletion, stubbing, and deep recall as policy surfaces that applications must expose deliberately.

In production deployments, users should be able to inspect stored memories, delete individual memories, disable memory, and request deletion of cold records and stubs as well as hot records. Sensitive categories such as medical, legal, financial, identity, and location facts should require stricter extraction policy, shorter retention defaults, or explicit user confirmation depending on the application domain. When a user deletes a memory, deletion must propagate to consolidated summaries, association edges, supersession links, audit references where legally permissible, and cold or stub representations, not only the hot retrieval index. Cold storage and reversible consolidation should not be interpreted as a right to retain everything indefinitely: they are mechanisms for avoiding accidental information loss within a configured

dormancy window, not a substitute for consent, data minimisation, encryption, access control, audit logging, and retention policy.

Deep recall is also a user-control issue. Because it intentionally widens retrieval to include cold and superseded records, applications should reserve it for explicit user requests, audit workflows, or agent actions with clear justification. A consumer-facing agent should not silently use deep recall to surface sensitive or previously superseded memories in ordinary conversation. The benchmark harness exercises deep recall as an architectural capability; deployment policy must decide when that capability is appropriate.

8 Limitations and Open Questions

We are candid about the current limitations of this work:

Most parameter choices remain heuristic. The v0.5 SDK release empirically tunes three defaults: `associative_boost` moves from 0.03 to 0.05, semantic base decay from 120 to 240 days, and `core_session_threshold` from 3 to 2. These changes are grounded in the public campaign logs listed in Code Availability and validated by the full-LoCoMo head-to-head in Table 4. Other values (the direct boost of 0.1, decay floors of 2% and 60%, core promotion stability/access thresholds, the $0.5\times$ deep recall penalty, the 90-day association decay constant, and the consolidation similarity threshold of 0.7) remain informed guesses based on cognitive science analogies and manual tuning. This paper is an architecture description; a subsequent empirical study using production audit logs will examine whether these values are reasonable and where they need adjustment.

Deferred conflict handling. Conflict handling is implemented as a queued maintenance process rather than an inline ingestion step. During ingestion, highly similar cross-session memories are queued as conflict candidates; `tick()` processes the queue with an LLM classifier using FadeMem’s contradiction/update/overlap taxonomy [Wei et al., 2026]. For contradictions or updates, the older record is linked to the newer one via `contradictedBy`; if the older record was core, its category is demoted so its floor drops from 60% to 2%. This avoids the $O(N^2)$ inline LLM bottleneck we observed during early LoCoMo runs, while still allowing corrected facts to supersede stale ones. The limitation is operational rather than conceptual: conflict resolution depends on maintenance cadence and queue processing, so a newly contradicted fact may persist until the next tick.

Limited statistical power across the empirical evaluation. Three caveats stack on the benchmark numbers. We report single-seed runs per condition, matching the practice of concurrent systems we compare against [Chhikara et al., 2025, Wei et al., 2026, Patel and Patel, 2025] but not characterising variance. LongMemEval is evaluated on the Small variant only (500 questions, ~ 53 haystack sessions per question); the Medium and Oracle variants would require multi-day wall-clock time and substantially higher API spend, and we treated LongMemEval-S as sufficient because it placed the system within measurement noise of the closest concurrent baseline at the time of running (ENGRAM, Patel and Patel 2025). Feature ablations (hybrid search, graph expansion, rerank, decay model) are run on LoCoMo conversation 0 only, not the full ten-conversation corpus, so effect signs and rough magnitudes are likely informative but precise per-feature pp values are not. A later protocol-split LoCoMo check completed one frozen held-out row with bootstrap intervals, but the corresponding benchmark-spec held-out ablations and LongMemEval-S transfer run remain deferred under available API budget. The supplemental local-model architecture-control rows cover the five-conversation held-out split, but they are off-spec checks under a local chat model and still single-seed; they do not replace benchmark-spec full-corpus ablations. The retrieval-evidence annotations in Section 6.7 are

LLM-judged rather than human-labeled, and the NaiveRAG comparison is one in-house run without confidence intervals. Multi-seed runs and full-corpus ablations are planned (Section 9).

Controlled LTI-Bench is small-sample and synthetic. Our LTI-Bench harness uses a hand-authored 30-day scenario with 28 facts and 42 probes across 8 probe types (Section 6.10). The scenario gives controlled access to the architectural properties the paper claims (direct decay probes, emergent core, conflict-driven supersession, time-aware retrieval), but Phase 8 shows that the direct decay probes do not isolate floor-clamping as the responsible mechanism. The per-category sample sizes ($n=4-8$) are too small to support strong statistical claims about general-purpose conversational performance. We treat LTI-Bench as a confirmatory architectural test, not a generalisation benchmark.

Consolidation clustering is underspecified. As noted in Section 3.8, the consolidation mechanism clusters fading memories by semantic similarity and compresses groups into summaries. The current implementation computes pairwise cosine similarity between embeddings of fading memories and groups those exceeding a threshold (default: 0.7); the grouping size of five is a configurable parameter chosen to balance summary coherence against information density. However, the LLM that performs compression lacks a principled framework for deciding what to preserve and what to discard. It is making editorial judgments about salience without ground truth, and the resulting summaries are lossy even though the originals are now retained in cold storage.

The preservation-first assumption is debatable. As noted in Section 3.9, our preservation-first policy follows the accessibility hypothesis: forgetting often reflects retrieval failure rather than storage loss [Tulving and Pearlstone, 1966]. There is also evidence for genuine *intrinsic forgetting*, where active molecular mechanisms degrade memory traces [Davis and Zhong, 2017]. Some memories may be erased, not merely made inaccessible. Our policy is pragmatic: text storage is cheap, information loss is irreversible, and the SDK can separate retention decay from storage policy. Decay floors are intended to prevent score collapse, cold migration controls index size, and a configurable cold-storage TTL can convert very old cold records into stubs.

Core memory thresholds remain unsettled. The thresholds for emergent core promotion (10 accesses, stability ≥ 0.85 , and a configurable cross-session threshold) are only partially empirically validated. Phase 2 suggests the old threshold of 3 sessions underperforms thresholds of 1–2 in the joint search, and v0.5 ships a threshold of 2. Too low, however, and trivial memories get promoted; too high, and genuinely important memories remain unprotected. LTI-Bench makes this risk concrete: 66 of 85 stored memories (78%) become core by day 30. That is likely too aggressive for a production memory system and should be treated as a threshold-tuning problem, not a desirable outcome.

Scalability of associative graph. The associative linking mechanism could produce a dense graph over time. Association weight decay (Section 3.7) mitigates unbounded growth by allowing unused associations to fall below the retrieval threshold, but we have not yet characterised the scaling behaviour at production volumes or determined whether explicit pruning of weak associations is necessary.

9 Future Work

Our immediate priorities are:

1. **Longer-horizon decay-floor ablation.** The Phase 8 ablation (Section 6.10) was a null result on LTI-Bench’s 30-day window: setting both `core` and `regular` floors to zero left `critical_fact_retention` and `summary_decay_trivial_mean_f1` unchanged at the bench’s reported precision. This rules out the floor mechanism being the dominant cause of those scores on this distribution, but does not test the architectural claim at horizons past 30 days, where stability decay would compound further. A 90-day or 180-day variant with controlled access patterns would close that gap.
2. **Production-data validation and privacy review.** Using audit logs from `blah.chat` to validate or invalidate parameter choices only after appropriate consent, minimisation, and privacy review: do memories actually follow Ebbinghaus-like curves under real usage; how many retrievals does it take for stability to plateau; what is the revival rate for faint memories.
3. **Multi-seed and full-corpus ablations.** Re-running the ablation conditions in Section 6 across multiple seeds and across the full ten-conversation LoCoMo corpus, to convert the current point-estimate effect sizes into population estimates. The off-spec local-model held-out rows are complete, but the benchmark-spec ablation rows remain deferred under current API-budget constraints.
4. **Cross-benchmark validation of tuned defaults.** Phase 5 validates v0.5 defaults on full LoCoMo. LongMemEval-S should be rerun as a v0.4/v0.5 head-to-head before claiming the same defaults generalise across benchmark families; the LoCoMo-dev-selected frozen config should also be transferred unchanged once budget permits.
5. **Strengthening associative retrieval.** The LTI-Bench results (Section 6.10) flag associative retrieval as the weakest probe category: cross-fact queries return a subset of the relevant cluster rather than the full cluster. Candidate fixes include explicit graph-walk expansion at retrieval time, broader top- k for associative-typed queries, and a query classifier that routes “what do you know about X ” style queries to a different retrieval policy.
6. **Cross-model generalisation.** The benchmark-spec rows use gpt-4o-mini as the answer model and gpt-4o-2024-08-06 as the judge, while the supplemental local-model rows disable the judge and use `openai/gpt-oss-120b`. We have not characterised how the system performs with Claude or Llama-family answer models, or whether the judge-model choice itself influences relative rankings.
7. **LongMemEval-M and Oracle.** Extending evaluation to the larger LongMemEval variants for a stronger comparison against recent systems such as TiMem, EverMemOS, MAGMA, Hindsight, and All-Mem.

10 Conclusion

Forgetting in agent memory is not unsolved, but it is unsettled. Multiple systems now implement decay mechanisms inspired by cognitive science, and no single approach has emerged as dominant. This paper contributes to that active conversation by describing an architecture with several specific design commitments: that memories should not be pruned simply because they are currently inaccessible, that core status should be earnable through use rather than only assignable at extraction, that retrieval should produce differentiated reinforcement depending on whether access was direct or associative, and that associative links between memories should themselves be dynamic.

These commitments are grounded in specific cognitive science mechanisms, implemented in open-source software, and deployed in a production application. Section 6 provides a first empirical pass. On LoCoMo, tuned v0.5 defaults reach 46.2% overall F1 and 51.3% multi-hop F1, improving over the paper-faithful v0.4 row by +1.87pp F1 and +2.73pp LLM-judge accuracy. On LongMemEval-S, task-averaged accuracy is 71.6%, competitive with concurrent single-stage memory systems but behind higher-scoring or differently configured architectures published around the same window. The controlled long-term-interaction harness shows 100% retention of identity-critical facts across a 30-day mixed-access window, compared with FadeMem’s reported 82.1% critical retention on a similar synthetic decay scenario, while also exposing likely over-promotion to core status. The Phase 8 decay-floor ablation rules out the floor clamp as the dominant mechanism on this distribution: setting floors to zero leaves 30-day critical retention unchanged, so the benchmark evidence supports the broader preservation-first lifecycle rather than floor-clamping in isolation. The numbers do not unambiguously settle whether decay floors and emergent core promotion improve downstream task performance over alternative architectures; they do indicate that the architecture is competitive at single-stage memory retrieval and that several lifecycle commitments behave as designed under controlled tests.

The later mechanism checks make that conclusion narrower and clearer. Direct evidence annotations show the full architecture ranking supporting evidence higher than vector-only and targeted ablations, while the NaiveRAG baseline shows the largest downstream advantage concentrated on temporal questions and a cost on simple lookups. The contribution is therefore not "more machinery always helps"; it is that lifecycle-aware retrieval changes which evidence remains accessible and where it appears in the ranking.

Code Availability

The `cognitive-memory` package and the simulation code used to generate Figures 1, 2, and 3 are available at:

- SDK (TypeScript and Python): <https://github.com/planetariescape/cognitive-memory>
- Benchmarks code and per-run JSON results:
<https://github.com/planetariescape/cognitive-memory-benchmarks>
- Paper source and PDF:
<https://github.com/planetariescape/cognitive-memory-benchmarks/tree/main/paper>
- Main LoCoMo artifact: `tuning/runs/phase5/v05_tuned/aggregate.json` and per-conversation `conv*.json`
- LoCoMo v0.4/v0.5 comparison: `tuning/runs/phase5/summary.json`
- Protocol-split LoCoMo artifacts: `tuning/splits/peer_review_20260511/`, `tuning/runs/locomo-0012/run-00/result.json`, and `docs/milestones/phase-10-peer-review-heldout.md`
- Supplemental local-model LoCoMo architecture-control artifacts: `tuning/runs/locomo-0026/`, `locomo-0029/`, `locomo-0030/`, `locomo-0031/`, `locomo-0032/`, `locomo-0033/`, `locomo-0034/`, `locomo-0035/`, `tuning/runs/phase12-local-architecture-control/paired_deltas_all_rows.md`, and `docs/milestones/phase-12-local-architecture-control.md`
- Retrieval-evidence control artifacts: `phase13-retrieval-evidence/` outputs and `docs/milestones/phase-13-retrieval-evidence-plan.md`

- Temporal reconstruction A/B artifacts: `phase14-temporal-reconstruction/` outputs and `docs/milestones/phase-14-temporal-reconstruction.md`
- NaiveRAG LoCoMo baseline artifact: `locomo/results/naive_rag/` outputs and `docs/milestones/phase-15-naive-rag-baseline.md`
- Main LongMemEval-S artifact: `longmemeval/results/current_sdk_20260505/primary.json`
- Main LTI-Bench artifact: `lti/results/current_sdk_20260505/run_1_v2.json`
- Phase 8 decay-floor ablation artifacts: `tuning/runs/lti-0109/`, `tuning/runs/lti-0110/`, and `docs/milestones/phase-8-decay-floor-ablation.md`
- Artifact commits: Phase 5 LoCoMo uses benchmark commit `82f08c2` and SDK commit `707758d`; Phase 8 uses benchmark commit `4ffae9` and SDK commit `7da1467`
- Experiment provenance: `experimentlog_v2.md`, `docs/milestones/`, `tuning/runs/runs.jsonl`, and per-trial `tuning/runs/lti-*`
- npm package: `cognitive-memory` (latest public release 0.4.0)
- PyPI package: `cognitive-memory` (latest public release 0.5.1)

The evaluation in Section 6 uses the May 2026 current-refresh artifacts, the Phase 5 full-LoCoMo v0.4/v0.5 head-to-head, the Phase 8 decay-floor ablation, and the later protocol-split LoCoMo held-out check recorded in the benchmarks repository. LongMemEval-S and the main LTI-Bench table remain current-refresh results; the LoCoMo headline row is the later v0.5 tuned-default result; the floors-off result is Phase 8. The supplemental local-model architecture control, retrieval-evidence control, temporal reconstruction A/B, and NaiveRAG baseline are reported as mechanism and positioning evidence rather than benchmark-spec headline replacements.

References

- Robert A. Bjork and Elizabeth L. Bjork. A new theory of disuse and an old theory of stimulus fluctuation. *From Learning Processes to Cognitive Processes: Essays in Honor of William K. Estes*, 2:35–67, 1992.
- Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- Prateek Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Allan M. Collins and Elizabeth F. Loftus. A spreading-activation theory of semantic processing. *Psychological Review*, 82(6):407–428, 1975.
- Ronald L. Davis and Yi Zhong. The biology of forgetting—a perspective. *Neuron*, 95(3):490–503, 2017.
- Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University, 1885. Translated by H. A. Ruger and C. E. Bussenius, 1913.
- Jason R. Finley. Expanded taxonomies of human memory. *Frontiers in Cognition*, 2025.

- Uwe Frey and Richard G. M. Morris. Synaptic tagging and long-term potentiation. *Nature*, 385:533–536, 1997.
- Axel Guskjolen et al. Natural forgetting reversibly modulates engram expression. *eLife*, 13: e92860, 2024.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. *arXiv preprint arXiv:2601.02163*, 2026. <https://arxiv.org/abs/2601.02163>.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. MAGMA: A multi-graph based agentic memory architecture for AI agents. *arXiv preprint arXiv:2601.03236*, 2026. <https://arxiv.org/abs/2601.03236>.
- Jiazheng Kang, Mingyu Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025.
- Chris Latimer, Nicolás Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025. <https://arxiv.org/abs/2512.12818>.
- Jaewoo Lee and Bosung Jang. Memoria: Resolving fateful forgetting problem through human-inspired memory architecture. *arXiv preprint arXiv:2310.03052*, 2023.
- Kai Li, Xuanqing Yu, Ziyi Ni, Yi Zeng, Yao Xu, Zheqing Zhang, Xin Li, Jitao Sang, Xiaogang Duan, Xuele Wang, Chengbao Liu, and Jie Tan. TiMem: Temporal-hierarchical memory consolidation for long-horizon conversational agents. *arXiv preprint arXiv:2601.02845*, 2026. <https://arxiv.org/abs/2601.02845>.
- Xuechen Liang, Yangfan He, Yinghui Xia, Xinyuan Song, Jianhui Wang, Meiling Tao, Li Sun, Xinhang Yuan, Jiayi Su, Keqin Li, Jiaqi Chen, Jinsong Yang, Siyuan Chen, and Tianyu Shi. SAGE: Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872*, 2024.
- Shichun Liu et al. Memory in the age of AI agents: A survey. *arXiv preprint arXiv:2512.13564*, 2025.
- Can Lv, Heng Chang, Yuchen Guo, Shengyu Tao, and Shiji Zhou. All-Mem: Agentic lifelong memory via dynamic topology evolution. *arXiv preprint arXiv:2603.19595*, 2026. <https://arxiv.org/abs/2603.19595>.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024. <https://arxiv.org/abs/2402.17753>.
- James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3): 419–457, 1995.
- Jaap M. J. Murre and Joeri Dros. Replication and analysis of Ebbinghaus’ forgetting curve. *PLoS ONE*, 10(7):e0120644, 2015.

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*. ACM, 2023.
- Daivik Patel and Shrenik Patel. ENGRAM: Effective, lightweight memory orchestration for conversational agents. *arXiv preprint arXiv:2511.12960*, 2025. <https://arxiv.org/abs/2511.12960>.
- Vencislav Popov, Iva Marevic, Jan Rummel, and Lynne M. Reder. Forgetting is a feature, not a bug: Intentionally forgetting some things is an effortful process that aids memory for target items. *Psychological Science*, 30(9):1303–1315, 2019.
- Blake A. Richards and Paul W. Frankland. The persistence and transience of memory. *Neuron*, 94(6):1071–1084, 2017.
- Tomás J. Ryan and Paul W. Frankland. Forgetting as a form of adaptive engram cell plasticity. *Nature Reviews Neuroscience*, 23:173–186, 2022.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Endel Tulving and Zena Pearlstone. Availability versus accessibility of information in memory for words. *Journal of Verbal Learning and Verbal Behavior*, 5(4):381–391, 1966.
- Lei Wei, Xu Dong, Xiao Peng, Niantao Xie, and Bin Wang. Fademem: Biologically-inspired forgetting for efficient agent memory. *arXiv preprint arXiv:2601.18642*, 2026.
- John T. Wixted. The psychology and neuroscience of forgetting. *Annual Review of Psychology*, 55:235–269, 2004.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. <https://arxiv.org/abs/2410.10813>.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

Two-Tier Retrieval Boosting: Direct vs. Associative

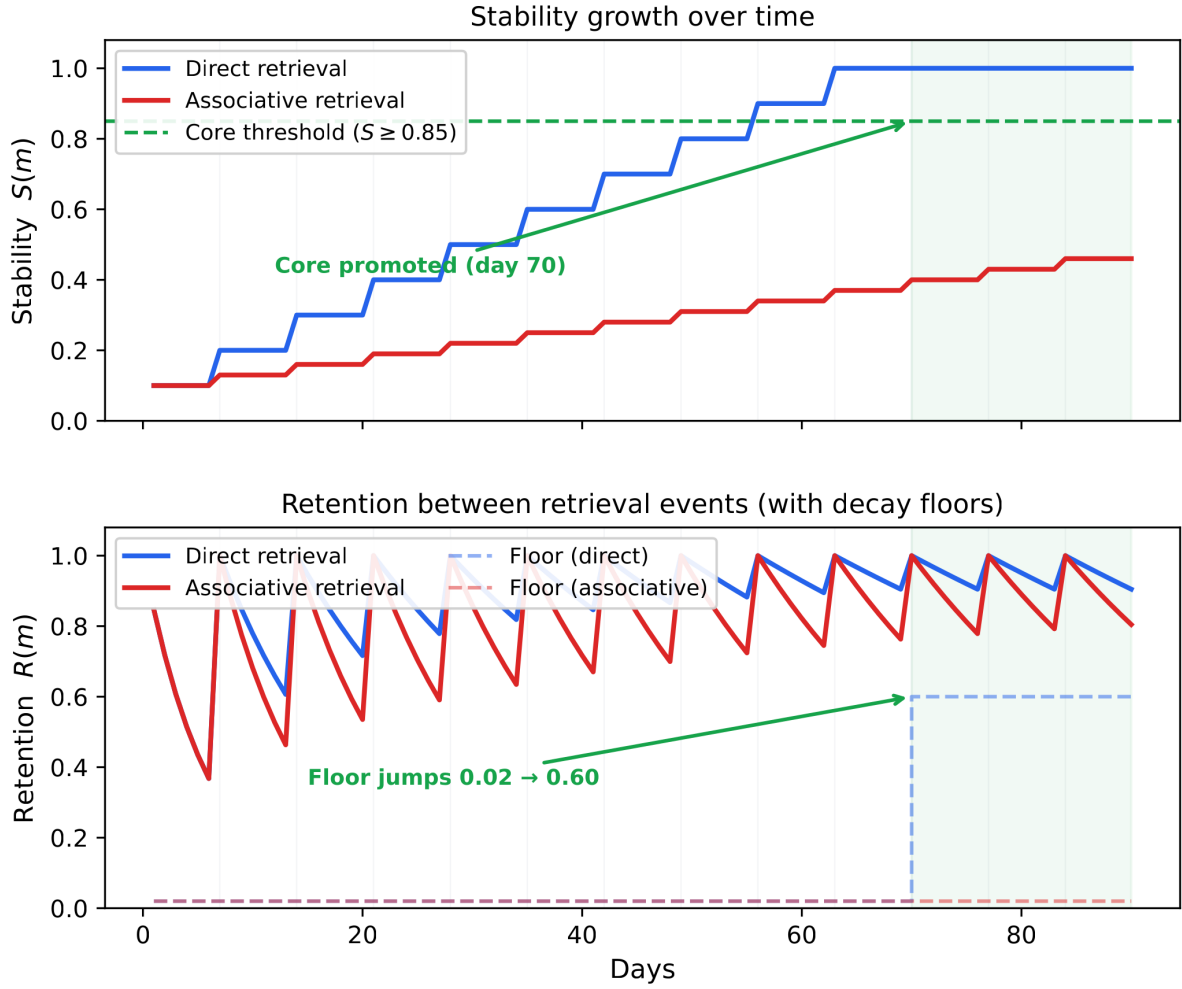


Figure 1: Simulated divergence between direct and associative retrieval over 90 days. Both memories start with identical parameters ($S = 0.1$, importance = 0.5, episodic category) and are retrieved every 7 days. **Top:** Stability growth, with the core promotion threshold ($S \geq 0.85$) marked. The directly retrieved memory crosses at day 70; the associative one peaks at $S = 0.46$. **Bottom:** Retention between retrieval events. At day 70 the directly retrieved memory's decay floor jumps from 0.02 to 0.60 upon core promotion; the associative memory's floor stays at 0.02.

Monte Carlo Analysis: Two-Tier Boosting Under Randomised Retrieval

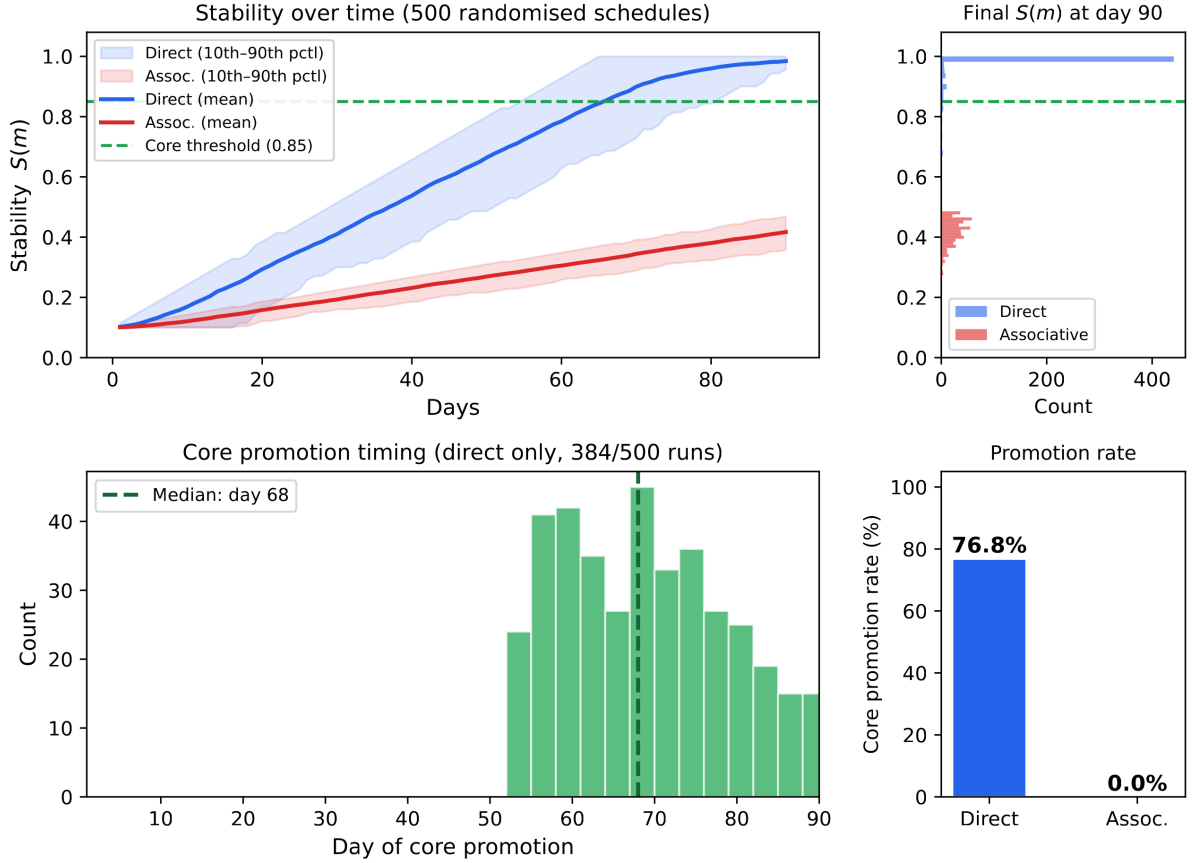


Figure 2: Monte Carlo analysis of two-tier boosting across 500 randomised retrieval schedules (~ 12 Poisson-distributed events per run, sessions every 14 days). **Top left:** Mean stability trajectories with 10th–90th percentile bands. The direct retrieval mean crosses the core threshold around day 60; the associative mean never approaches it. **Top right:** Distribution of final stability at day 90, showing near-complete separation between the two tiers. **Bottom left:** Distribution of core promotion timing for direct retrieval (median: day 68). **Bottom right:** Core promotion rates: 76.8% for direct, 0% for associative.

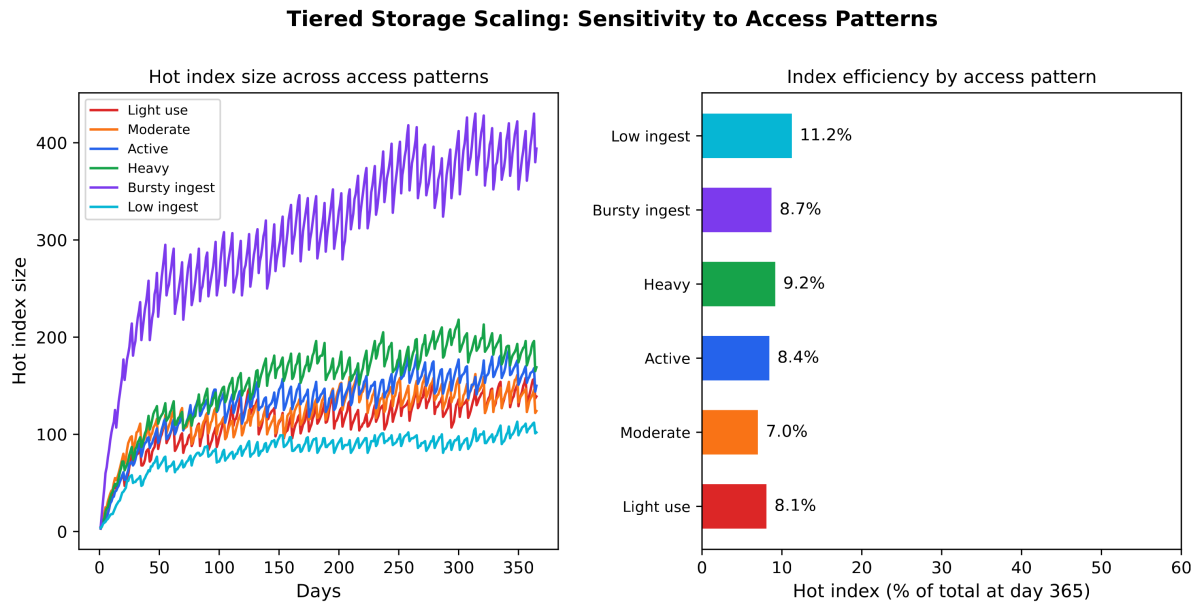


Figure 3: Tiered storage scaling across six access patterns over one simulated year. **Left:** Hot index size over time. All scenarios plateau; absolute size scales with ingestion rate but growth converges to zero. The bursty ingest scenario (10 memories/day, 2 queries/day) shows the most variance but still stabilises. **Right:** Hot index as percentage of total memories at day 365. All scenarios converge to the 7–11% range, indicating that bounded hot index size is robust to access pattern variation.